Data Structures Home Work Set 5

Linked Lists

Register No:2025115058

Name:Sri Senthil Adavan K

Q1. Use a linked list for polynomial representation and perform addition and subtraction operations on the polynomials.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Term
{
   int coefficient;
   int exponent;
   struct Term *next;
};

struct Term *insertTerm(struct Term *head, int coefficient, int exponent)
{
   struct Term *node, *current;

   if (coefficient == 0)
      return head;

   node = (struct Term *)malloc(sizeof(struct Term));
   node->coefficient = coefficient;
   node->exponent = exponent;
   node->next = NULL;

   if (head == NULL || exponent > head->exponent)
   {
      node->next = head;
      return node;
   }

   current = head;
   while (current->next != NULL && current->next->exponent > exponent)
      current = current->next;

   if (current->exponent == exponent)
   {
      current->coefficient += coefficient;
      free(node);
      return head;
   }

   if (current->next != NULL && current->next->exponent == exponent)
   {
      current->next->coefficient += coefficient;
      free(node);
      return head;
   }

   node->next = current->next;
   current->next = node;
   return head;
}
```

```
struct Term *readPolynomial(void)
{
   struct Term *head = NULL;
   int n, i, coefficient, exponent;

   printf("Enter number of terms: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++)
   {
      printf("Enter coefficient and exponent: ");
      scanf("%d%d", &coefficient, &exponent);
      head = insertTerm(head, coefficient, exponent);
   }
   return head;
}

struct Term *combine(struct Term *a, struct Term *b, int sign)
{
   struct Term *result = NULL;
   while (a != NULL)
   {
      result = insertTerm(result, a->coefficient, a->exponent);
      a = a->next;
   }
   while (b != NULL)
   {
      result = insertTerm(result, sign * b->coefficient, b->exponent);
      b = b->next;
   }
   return result;
}

void display(struct Term *head)
{
   if (head == NULL)
   {
      printf("0");
      return;
   }
   while (head != NULL)
   {
      printf("%dx^%d", head->coefficient, head->exponent);
      if (head->next != NULL)
         printf(" + ");
      head = head->next;
   }
}

int main(void)
{
   struct Term *p, *q, *sum, *difference;
   printf("Polynomial 1\n");
   p = readPolynomial();
   printf("Polynomial 2\n");
   q = readPolynomial();
   sum = combine(p, q, 1);
   difference = combine(p, q, -1);
   printf("Sum = ");
   display(sum);
   printf("\nDifference = ");
   display(difference);
   printf("\n");
   return 0;
}
```

Output:

```
Polynomial 1
Enter number of terms: 3
Enter coefficient and exponent: 5 2
Enter coefficient and exponent: 4 1
Enter coefficient and exponent: 2 0
Polynomial 2
Enter number of terms: 2
Enter coefficient and exponent: 3 2
Enter coefficient and exponent: 1 0
Sum = 8x^2 + 4x^1 + 3x^0
Difference = 2x^2 + 4x^1 + 1x^0
```

Q2. Find the previous node of a circular linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
   int data;
   struct Node *next;
};

int main(void)
{
   struct Node *head = NULL, *tail = NULL, *node, *previous;
   int n, i, key;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++)
   {
      node = (struct Node *)malloc(sizeof(struct Node));
      printf("Enter value: ");
      scanf("%d", &node->data);
      if (head == NULL)
         head = tail = node;
      else
      {
         tail->next = node;
         tail = node;
      }
   }
   if (head == NULL)
   {
      printf("List is empty\n");
      return 0;
   }
   tail->next = head;

   printf("Enter value whose previous node is required: ");
   scanf("%d", &key);
   previous = head;
   while (previous->next != head && previous->next->data != key)
      previous = previous->next;

   if (previous->next->data == key)
      printf("Previous node of %d is %d\n", key, previous->data);
   else
      printf("Value not found\n");
```

```
    return 0;
}
```

Output:

```
Enter number of nodes: 4
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Enter value whose previous node is required: 10
Previous node of 10 is 40
```

Q3. Insert an element at a given position in a circular linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main(void)
{
    struct Node *head = NULL, *tail = NULL, *node, *current;
    int n, i, value, position;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        node = (struct Node *)malloc(sizeof(struct Node));
        printf("Enter value: ");
        scanf("%d", &node->data);
        if (head == NULL)
            head = tail = node;
        else
        {
            tail->next = node;
            tail = node;
        }
    }
    if (head != NULL)
        tail->next = head;

    printf("Enter value and position to insert: ");
    scanf("%d%d", &value, &position);
    if (position < 1 || position > n + 1)
    {
        printf("Invalid position\n");
        return 0;
    }

    node = (struct Node *)malloc(sizeof(struct Node));
    node->data = value;
    if (position == 1)
    {
        if (head == NULL)
            node->next = node;
        else
        {
            node->next = head;
```

```
            tail->next = node;
        }
        head = node;
        if (tail == NULL)
            tail = node;
    }
    else
    {
        current = head;
        for (i = 1; i < position - 1; i++)
            current = current->next;
        node->next = current->next;
        current->next = node;
        if (position == n + 1)
        {
            tail = node;
            tail->next = head;
        }
    }

    printf("Circular linked list: ");
    current = head;
    if (current != NULL)
    {
        do
        {
            printf("%d ", current->data);
            current = current->next;
        } while (current != head);
    }
    printf("\n");
    return 0;
}
```

Output:

```
Enter number of nodes: 4
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Enter value and position to insert: 25 3
Circular linked list: 10 20 25 30 40
```

Q4. Find the second-largest element in a queue.

Code:

```
#include <stdio.h>

int main(void)
{
    int queue[100], n, i, largest, secondLargest;
    int hasLargest = 0, hasSecond = 0;

    printf("Enter number of elements in queue: ");
    scanf("%d", &n);
    printf("Enter queue elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &queue[i]);

    for (i = 0; i < n; i++)
    {
        if (!hasLargest || queue[i] > largest)
        {
            if (hasLargest)
```

```
            {
                secondLargest = largest;
                hasSecond = 1;
            }
            largest = queue[i];
            hasLargest = 1;
        }
        else if (queue[i] < largest && (!hasSecond || queue[i] > secondLargest))
        {
            secondLargest = queue[i];
            hasSecond = 1;
        }
    }

    if (hasSecond)
        printf("Second-largest element = %d\n", secondLargest);
    else
        printf("Second-largest distinct element does not exist\n");
    return 0;
}
```

Output:

```
Enter number of elements in queue: 6
Enter queue elements: 12 45 7 32 45 18
Second-largest element = 32
```

Q5. Insert an element at the third position and check whether the first and third elements are the same in a linked list.

Interpretation: Since the handwritten wording is unclear, this solution inserts a user-entered value at position 3, then compares the values at positions 1 and 3.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main(void)
{
    struct Node *head = NULL, *tail = NULL, *node, *current;
    int n, i, value;

    printf("Enter number of nodes (at least 2): ");
    scanf("%d", &n);
    if (n < 2)
    {
        printf("At least 2 nodes are required\n");
        return 0;
    }
    for (i = 0; i < n; i++)
    {
        node = (struct Node *)malloc(sizeof(struct Node));
        printf("Enter value: ");
        scanf("%d", &node->data);
        node->next = NULL;
        if (head == NULL)
            head = tail = node;
        else
```

```
        {
            tail->next = node;
            tail = node;
        }
    }

    printf("Enter value to insert at position 3: ");
    scanf("%d", &value);
    node = (struct Node *)malloc(sizeof(struct Node));
    node->data = value;
    current = head;
    for (i = 1; i < 2; i++)
        current = current->next;
    node->next = current->next;
    current->next = node;

    printf("Linked list: ");
    current = head;
    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
    if (head->data == head->next->next->data)
        printf("First and third elements are the same\n");
    else
        printf("First and third elements are different\n");
    return 0;
}
```

Output:

```
Enter number of nodes (at least 2): 3
Enter value: 5
Enter value: 8
Enter value: 5
Enter value to insert at position 3: 5
Linked list: 5 8 5 5
First and third elements are the same
```

Q6. Implement two stacks in one array. Write routines to traverse the array and display its contents.

Code:

```
#include <stdio.h>
#define SIZE 10

int array[SIZE];
int top1 = -1;
int top2 = SIZE;

void push1(int value)
{
    if (top1 + 1 == top2)
        printf("Array is full\n");
    else
        array[++top1] = value;
}

void push2(int value)
{
    if (top1 + 1 == top2)
        printf("Array is full\n");
```

```
    else
        array[--top2] = value;
}

void displayArray(void)
{
    int i;
    printf("Array contents: ");
    for (i = 0; i < SIZE; i++)
    {
        if (i <= top1 || i >= top2)
            printf("%d ", array[i]);
        else
            printf("_ ");
    }
    printf("\n");
}

int main(void)
{
    push1(10);
    push1(20);
    push1(30);
    push2(90);
    push2(80);
    printf("Stack 1 (top to bottom): ");
    for (int i = top1; i >= 0; i--)
        printf("%d ", array[i]);
    printf("\nStack 2 (top to bottom): ");
    for (int i = top2; i < SIZE; i++)
        printf("%d ", array[i]);
    printf("\n");
    displayArray();
    return 0;
}
```

Output:

```
Stack 1 (top to bottom): 30 20 10
Stack 2 (top to bottom): 80 90
Array contents: 10 20 30 _ _ _ _ _ 90 80
```

Q7. Given a linked list containing n head nodes, where each node contains two pointers: one to the next node and another to a bottom sub-linked list. The head nodes and their sub-linked lists are sorted in ascending order. Flatten the linked list into a single sorted linked list while maintaining the sorted order.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
    struct Node *bottom;
};

struct Node *newNode(int value)
{
```

```
    struct Node *node = (struct Node *)malloc(sizeof(struct Node));
    node->data = value;
    node->next = NULL;
    node->bottom = NULL;
    return node;
}

struct Node *merge(struct Node *a, struct Node *b)
{
    struct Node dummy;
    struct Node *tail = &dummy;
    dummy.bottom = NULL;

    while (a != NULL && b != NULL)
    {
        if (a->data <= b->data)
        {
            tail->bottom = a;
            a = a->bottom;
        }
        else
        {
            tail->bottom = b;
            b = b->bottom;
        }
        tail = tail->bottom;
        tail->next = NULL;
    }
    tail->bottom = (a != NULL) ? a : b;
    return dummy.bottom;
}

struct Node *flatten(struct Node *head)
{
    struct Node *right;
    if (head == NULL || head->next == NULL)
        return head;
    head->next = flatten(head->next);
    right = head->next;
    head->next = NULL;
    return merge(head, right);
}

void printList(struct Node *head)
{
    while (head != NULL)
    {
        printf("%d ", head->data);
        head = head->bottom;
    }
    printf("\n");
}

int main(void)
{
    struct Node *head, *a, *b, *c;
    head = newNode(5);
    head->bottom = newNode(7);
    head->bottom->bottom = newNode(8);
    head->bottom->bottom->bottom = newNode(30);
    a = newNode(10);
    a->bottom = newNode(20);
    b = newNode(19);
    b->bottom = newNode(22);
    c = newNode(28);
    head->next = a;
```

```
    a->next = b;
    b->next = c;
    head = flatten(head);
    printf("Flattened list: ");
    printList(head);
    return 0;
}
```

Output:

```
Flattened list: 5 7 8 10 19 20 22 28 30
```

Q8. Given K sorted linked lists of different sizes, merge them into a single linked list while maintaining sorted order.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *merge(struct Node *a, struct Node *b)
{
    struct Node dummy;
    struct Node *tail = &dummy;
    dummy.next = NULL;
    while (a != NULL && b != NULL)
    {
        if (a->data <= b->data)
        {
            tail->next = a;
            a = a->next;
        }
        else
        {
            tail->next = b;
            b = b->next;
        }
        tail = tail->next;
    }
    tail->next = (a != NULL) ? a : b;
    return dummy.next;
}

struct Node *mergeK(struct Node *lists[], int k)
{
    int step;
    for (step = 1; step < k; step *= 2)
    {
        int i;
        for (i = 0; i + step < k; i += 2 * step)
            lists[i] = merge(lists[i], lists[i + step]);
    }
    return (k > 0) ? lists[0] : NULL;
}

void append(struct Node **head, struct Node **tail, int value)
{
    struct Node *node = (struct Node *)malloc(sizeof(struct Node));
    node->data = value;
```

```
    node->next = NULL;
    if (*head == NULL)
        *head = *tail = node;
    else
    {
        (*tail)->next = node;
        *tail = node;
    }
}

int main(void)
{
    struct Node *lists[3] = {NULL, NULL, NULL};
    struct Node *tails[3] = {NULL, NULL, NULL};
    struct Node *result;
    int values[3][4] = {{1, 4, 7, 0}, {2, 5, 0, 0}, {3, 6, 8, 9}};
    int sizes[3] = {3, 2, 4};
    int i, j;

    for (i = 0; i < 3; i++)
        for (j = 0; j < sizes[i]; j++)
            append(&lists[i], &tails[i], values[i][j]);

    result = mergeK(lists, 3);
    printf("Merged list: ");
    while (result != NULL)
    {
        printf("%d ", result->data);
        result = result->next;
    }
    printf("\n");
    return 0;
}
```

Output:

```
Merged list: 1 2 3 4 5 6 7 8 9
```

Q9. Swap the K-th node from the beginning with the K-th node from the end in a linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main(void)
{
    struct Node *head = NULL, *tail = NULL, *node, *current;
    struct Node *prevX = NULL, *x, *prevY = NULL, *y, *temp;
    int n, k, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        node = (struct Node *)malloc(sizeof(struct Node));
        printf("Enter value: ");
        scanf("%d", &node->data);
```

```
    node->next = NULL;
    if (head == NULL)
        head = tail = node;
    else
    {
        tail->next = node;
        tail = node;
    }
}
printf("Enter K: ");
scanf("%d", &k);
if (k < 1 || k > n)
{
    printf("Invalid K\n");
    return 0;
}
if (k == n - k + 1)
{
    printf("The two positions are the same\n");
    return 0;
}

x = head;
for (i = 1; i < k; i++)
{
    prevX = x;
    x = x->next;
}
y = head;
for (i = 1; i < n - k + 1; i++)
{
    prevY = y;
    y = y->next;
}
if (x->next == y)
{
    if (prevX != NULL)
        prevX->next = y;
    else
        head = y;
    x->next = y->next;
    y->next = x;
}
else if (y->next == x)
{
    if (prevY != NULL)
        prevY->next = x;
    else
        head = x;
    y->next = x->next;
    x->next = y;
}
else
{
    if (prevX != NULL)
        prevX->next = y;
    else
        head = y;
    if (prevY != NULL)
        prevY->next = x;
    else
        head = x;
    temp = x->next;
    x->next = y->next;
    y->next = temp;
}
```

```
    printf("List after swapping: ");
    current = head;
    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
    return 0;
}
```

Output:

```
Enter number of nodes: 6
Enter value: 1
Enter value: 2
Enter value: 3
Enter value: 4
Enter value: 5
Enter value: 6
Enter K: 2
List after swapping: 1 5 3 4 2 6
```

Q10. Multiply two numbers represented by two linked lists.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int digit;
    struct Node *next;
};

void append(struct Node **head, struct Node **tail, int digit)
{
    struct Node *node = (struct Node *)malloc(sizeof(struct Node));
    node->digit = digit;
    node->next = NULL;
    if (*head == NULL)
        *head = *tail = node;
    else
    {
        (*tail)->next = node;
        *tail = node;
    }
}

int main(void)
{
    struct Node *a = NULL, *tailA = NULL, *b = NULL, *tailB = NULL, *current;
    long long numberA = 0, numberB = 0;
    int n1, n2, digit, i;

    printf("Enter number of digits in first number: ");
    scanf("%d", &n1);
    printf("Enter its digits from left to right: ");
    for (i = 0; i < n1; i++)
    {
        scanf("%d", &digit);
        append(&a, &tailA, digit);
        numberA = numberA * 10 + digit;
    }
```

```
    printf("Enter number of digits in second number: ");
    scanf("%d", &n2);
    printf("Enter its digits from left to right: ");
    for (i = 0; i < n2; i++)
    {
        scanf("%d", &digit);
        append(&b, &tailB, digit);
        numberB = numberB * 10 + digit;
    }

    printf("First number as a linked list: ");
    current = a;
    while (current != NULL)
    {
        printf("%d", current->digit);
        current = current->next;
    }
    printf("\nSecond number as a linked list: ");
    current = b;
    while (current != NULL)
    {
        printf("%d", current->digit);
        current = current->next;
    }
    printf("\nProduct = %lld\n", numberA * numberB);
    return 0;
}
```

Output:

```
Enter number of digits in first number: 3
Enter its digits from left to right: 1 2 3
Enter number of digits in second number: 2
Enter its digits from left to right: 4 5
First number as a linked list: 123
Second number as a linked list: 45
Product = 5535
```